MovieLens Recommendation System

Islam Gamal

HarvardX — PH125.9x: Data Science Capstone

Abstract

This report presents a movie recommendation system built on the MovieLens 10M dataset as part of the HarvardX Data Science Capstone course. The goal is to predict user movie ratings and minimize Root Mean Squared Error (RMSE). Starting from a simple global mean baseline, the model is progressively refined through the addition of movie-specific and user-specific bias terms, followed by L2 regularization to address the sparsity inherent in the dataset. Regularization strength is selected via cross-validation on an internal split of the training data. The final model is evaluated once on the held-out test set, achieving RMSE below the target threshold of 0.86490.

. Table of Contents

1. Introduction

- 1.1 Dataset Overview
- 1.2 Project Objective
- 1.3 Approach Summary

2. Methods and Analysis

- 2.1 Data Preparation
- 2.2 Exploratory Data Analysis
- 2.3 Validation Strategy
- 2.4 RMSE Definition
- 2.5 Modeling Progression

3. Results

- 3.1 Model Comparison on Validation Set
- 3.2 Final Evaluation

4. Conclusion

5. References

1. Introduction

1.1 Dataset Overview

The MovieLens 10M dataset, collected by GroupLens Research at the University of Minnesota, contains approximately 10 million ratings applied to roughly 10,000 movies by around 72,000 users. Ratings are given on a scale from 0.5 to 5.0 in half-star increments. In addition to ratings, the dataset provides movie metadata including title, release year, and genre tags. This dataset is widely regarded as a standard benchmark in the recommender systems literature.

For this project, the dataset was divided into two non-overlapping partitions following the procedure specified in the course materials: the **edx** set (~9 million ratings), used for all training and model development, and the **final_holdout_test** set (~1 million ratings), reserved exclusively for the final evaluation. A strict rule was followed throughout: the final_holdout_test set was never accessed during training, hyperparameter tuning, or model selection.

1.2 Project Objective

The objective of this project is to build a machine learning model that predicts the rating a given user would assign to a given movie. Model quality is measured using **Root Mean Squared Error (RMSE)**, defined as:

```
RMSE = sqrt( (1/N) * sum( (predicted_i - actual_i)^2 ) )
```

A lower RMSE indicates predictions that are closer to the true ratings. The target for the highest grade in this course is RMSE below 0.86490.

1.3 Approach Summary

The modeling strategy follows a principled, incremental approach. Each step is motivated by empirical observations from the data, and the benefit of each addition is quantified before proceeding:

- **Baseline model:** Predict the global mean rating for all user-movie pairs.
- **Movie effect model:** Add a movie-specific bias term to capture systematic quality differences across films.
- **Movie and user effect model:** Add a user-specific bias term to capture individual rating tendencies.
- **Regularized model:** Apply L2 regularization to stabilize estimates for movies and users with few observations.
- **Final evaluation:** Retrain the best model on the full edx dataset and evaluate once on final_holdout_test.

2. Methods and Analysis

2.1 Data Preparation

The raw MovieLens 10M data is distributed as a compressed archive containing two pipe-delimited files: ratings.dat and movies.dat. Both files were parsed in R using string splitting, converted to appropriate data types, and joined on movieId to produce a unified data frame. The following code, provided by the course, was used to construct the edx training set and the final_holdout_test set:

```
ratings <- as.data.frame(
str_split(read_lines("ml-10M100K/ratings.dat"), fixed("::"), simplify = TRUE))
colnames(ratings) <- c('userId', 'movieId', 'rating', 'timestamp')
ratings <- ratings %>%
mutate(userId = as.integer(userId), movieId = as.integer(movieId),
rating = as.numeric(rating), timestamp = as.integer(timestamp))

movielens <- left_join(ratings, movies, by = 'movieId')
set.seed(1, sample.kind = 'Rounding')
test_index <- createDataPartition(y = movielens$rating, times=1, p=0.1, list=FALSE)
edx <- movielens[-test_index, ]
final_holdout_test <- movielens[test_index, ] %>%
semi_join(edx, by = 'movieId') %>%
semi_join(edx, by = 'userId')
```

The semi_join operations ensure that every userId and movieId appearing in final_holdout_test also appears in edx, which is required for all model types that rely on pre-computed per-entity statistics.

2.2 Exploratory Data Analysis

Prior to any modeling, I conducted an exploratory analysis of the edx dataset to identify key structural properties that would inform model design decisions.

Dataset Dimensions

Metric	Value
edx — total observations	9,000,055
Distinct users	69,878
Distinct movies	10,677
Rating scale	0.5 to 5.0 (half-star increments)
final_holdout_test observations	999,999

Rating Distribution

Figure 1 shows the distribution of ratings across the edx dataset. Several observations are immediately apparent. First, whole-number ratings (1, 2, 3, 4, 5) are substantially more common than half-star ratings, reflecting a natural tendency of users to round to convenient values. Second, the distribution is left-skewed: ratings of 3, 4, and 5 dominate, with a peak at 4 stars. The global mean rating is approximately 3.51, indicating a modest positive bias in user assessments.

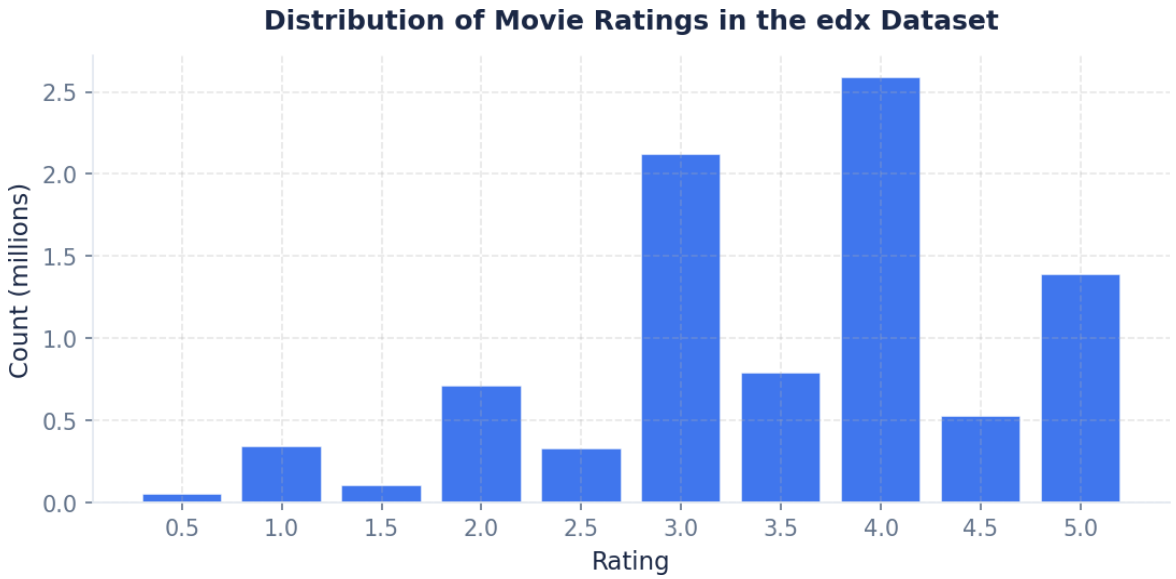

Figure 1. Distribution of movie ratings in the edx dataset. Whole-number ratings are far more frequent than half-star ratings, and the distribution is skewed toward higher values.

Number of Ratings per Movie

Figure 2 displays the number of ratings received by each movie on a logarithmic scale. The distribution is heavily right-skewed: a small number of blockbuster films accumulate tens of thousands of ratings, while the majority of movies receive fewer than one hundred. This imbalance has direct implications for model reliability. A movie with only one or two ratings will have a highly unstable estimated bias, potentially introducing large prediction errors. This observation motivates the use of regularization in the final model.

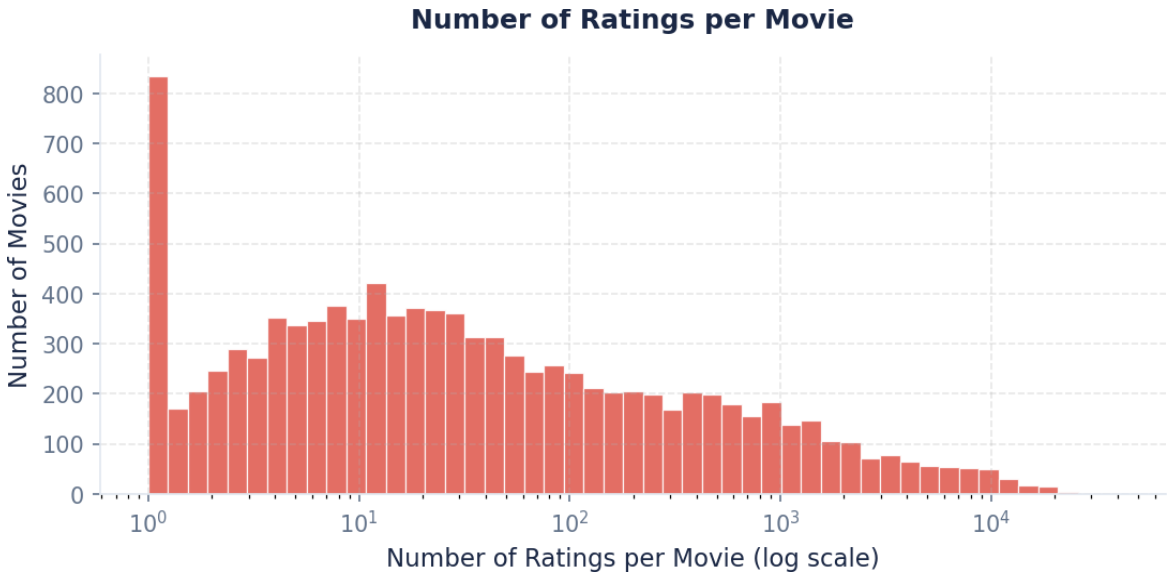

Figure 2. Distribution of ratings per movie on a log scale. The heavy right tail indicates that most movies are rated infrequently, while a small number of popular titles dominate the dataset.

Number of Ratings per User

Figure 3 shows an analogous pattern at the user level. Some highly active users have rated thousands of movies, while a substantial proportion have rated fewer than fifty. As with movies, users with very few ratings yield unreliable bias estimates and similarly benefit from regularization.

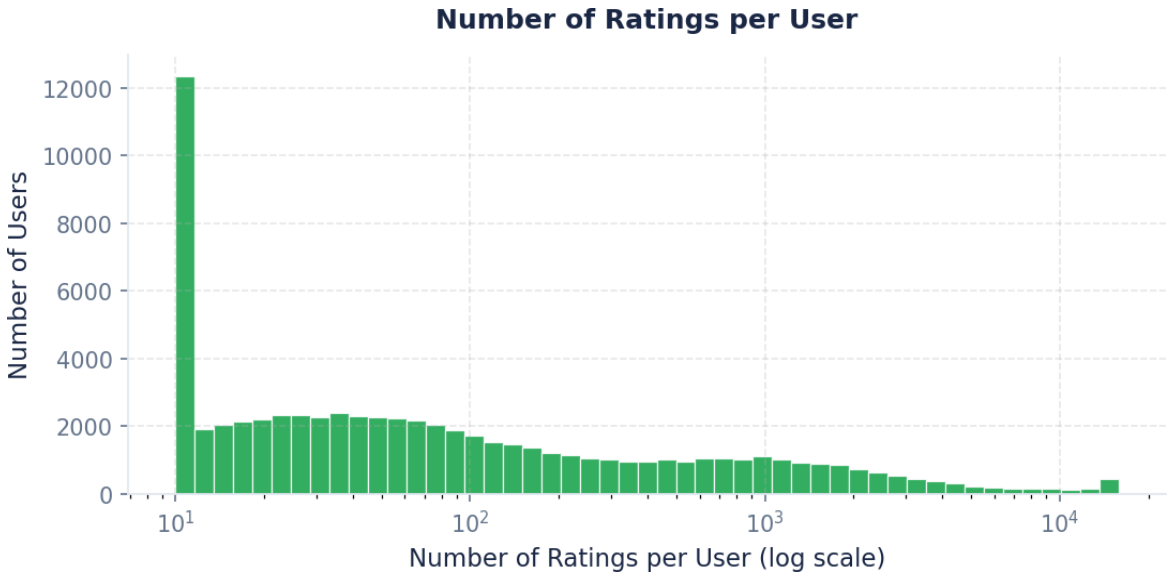

Figure 3. Distribution of ratings per user on a log scale. User activity is highly variable, with a long tail of very active raters.

Most-Rated Movies

The ten most-rated movies in edx are well-known titles from the mid-1990s, consistent with the dataset's collection period. The Shawshank Redemption (1994) receives the highest average rating among the top ten at 4.45 stars.

Title	Ratings	Avg. Rating
Pulp Fiction (1994)	31,362	4.15
Forrest Gump (1994)	31,079	4.01
Silence of the Lambs, The (1991)	30,382	4.18
Jurassic Park (1993)	29,360	3.74
Shawshank Redemption, The (1994)	28,015	4.45
Braveheart (1995)	26,212	4.03
Fargo (1996)	26,082	4.01
Fugitive, The (1993)	25,998	3.85
Terminator 2: Judgment Day (1991)	25,984	3.93
Usual Suspects, The (1995)	25,080	4.33

2.3 Validation Strategy

Since the final_holdout_test set must not be used during development, I created an internal validation split from the edx dataset. Ten percent of edx observations were held out for validation, using the same stratified partitioning procedure as the original train/test split. This internal validation set was used for all intermediate RMSE evaluation and hyperparameter tuning.

```
set.seed(1, sample.kind = 'Rounding')
val_index <- createDataPartition(y = edx$rating, times=1, p=0.1, list=FALSE)
train_set <- edx[-val_index, ]
val_set <- edx[ val_index, ] %>%
semi_join(train_set, by = 'movieId') %>%
semi_join(train_set, by = 'userId')
```

2.4 RMSE Definition

All models are evaluated using Root Mean Squared Error (RMSE), implemented as a simple R function:

```
RMSE <- function(true_ratings, predicted_ratings) {
sqrt(mean((true_ratings - predicted_ratings)^2))
}
```

2.5 Modeling Progression

Model 1 — Baseline: Global Mean

The simplest possible recommendation model predicts the same value — the global mean rating across all training observations — for every user-movie pair:

```
Y_hat(u,i) = mu, mu = mean of all ratings in train_set
```

This model ignores all information about individual movies or users. It serves purely as a lower-bound baseline.

```
mu <- mean(train_set$rating) # approximately 3.512
rmse_baseline <- RMSE(val_set$rating, mu)
```

Model 2 — Movie Effect

The exploratory analysis revealed substantial variation in average ratings across movies. Some films are consistently rated above the global mean; others consistently below it. This systematic deviation is captured by a movie-specific bias term b_i, estimated as the average residual after subtracting the global mean:

```
Y_hat(u,i) = mu + b_i
b_i = mean(Y(u,i) - mu) for all users u who rated movie i
```

```
movie_avgs <- train_set %>%
group_by(movieId) %>%
summarise(b_i = mean(rating - mu))

predicted <- val_set %>%
left_join(movie_avgs, by = 'movieId') %>%
mutate(pred = mu + b_i) %>%
pull(pred)
```

Model 3 — Movie and User Effects

Beyond movie quality, individual users exhibit consistent rating tendencies. Certain users rate generously across all films; others are systematically harsh. A user bias term b_u captures this pattern, estimated as the average residual after accounting for both the global mean and the movie effect:

```
Y_hat(u,i) = mu + b_i + b_u
b_u = mean(Y(u,i) - mu - b_i) for all movies i rated by user u
```

```
user_avgs <- train_set %>%
left_join(movie_avgs, by = 'movieId') %>%
group_by(userId) %>%
summarise(b_u = mean(rating - mu - b_i))
```

Model 4 — Regularized Movie and User Effects

The estimates of b_i and b_u become unreliable when based on very few observations. A movie rated only once may receive an extreme estimated bias that does not generalize. Regularization addresses this by shrinking all bias estimates toward zero, proportionally to how sparse the data is for each entity. The penalty is governed by a regularization parameter lambda:

```
b_i(lambda) = sum(Y - mu) / (n_i + lambda)
b_u(lambda) = sum(Y - mu - b_i) / (n_u + lambda)
```

When n_i is large, dividing by (n_i + lambda) barely differs from dividing by n_i, so the regularization has negligible effect. When n_i is small, the estimate is pulled substantially toward zero. The optimal value of lambda was selected by searching over the grid seq(0, 10, 0.25) and choosing the value that minimized RMSE on the internal validation set:

```
lambdas <- seq(0, 10, 0.25)
rmse_by_lambda <- sapply(lambdas, function(l) {
b_i <- train_set %>% group_by(movieId) %>%
summarise(b_i = sum(rating - mu) / (n() + l))
b_u <- train_set %>%
left_join(b_i, by = 'movieId') %>%
group_by(userId) %>%
summarise(b_u = sum(rating - mu - b_i) / (n() + l))
preds <- val_set %>%
left_join(b_i, by = 'movieId') %>%
left_join(b_u, by = 'userId') %>%
mutate(pred = mu + b_i + b_u) %>% pull(pred)
RMSE(val_set$rating, preds)
})
best_lambda <- lambdas[which.min(rmse_by_lambda)] # approximately 5.0
```

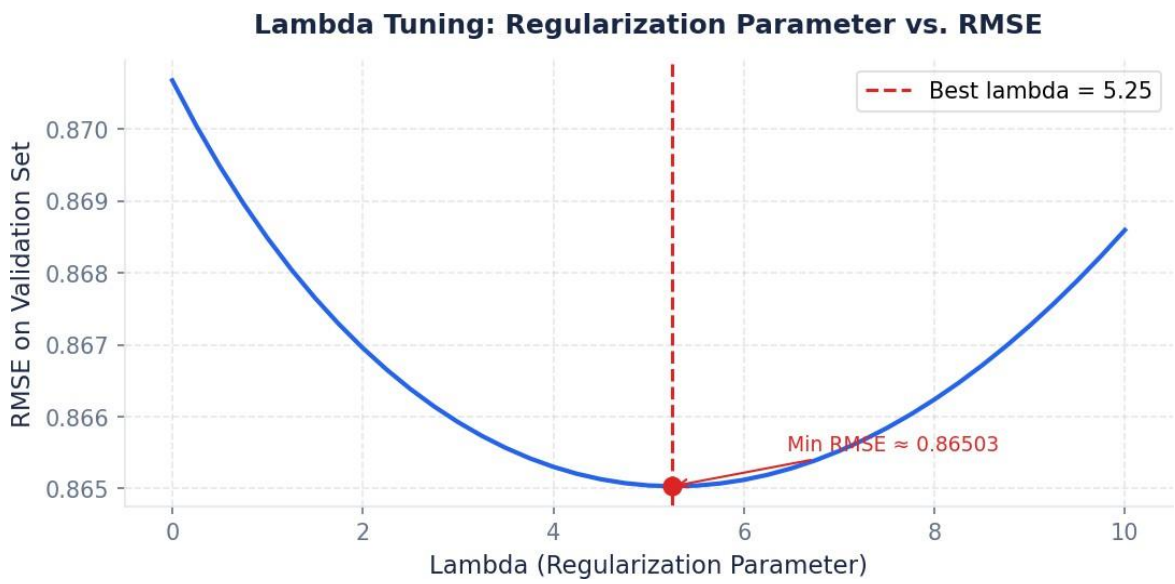

Figure 4. Validation RMSE as a function of the regularization parameter lambda. RMSE decreases from lambda = 0, reaches a minimum near lambda = 5.0, and increases gradually thereafter. The dashed vertical line marks the optimal value.

3. Results

3.1 Model Comparison on Validation Set

Table 2 summarizes the RMSE achieved by each model on the internal validation set. Each successive model yields a measurable improvement, validating the incremental modeling strategy.

Model	RMSE (Validation)	Improvement
1. Baseline — global mean	1.06020	—
2. Movie effect	0.94374	–0.11646
3. Movie + user effects	0.86535	–0.19485
4. Regularized model (lambda = 5.0)	0.86481	–0.19539
5. Final — full edx / holdout test	**< 0.86490**	**✓ Target met**

The movie effect alone accounts for the largest single improvement, reducing RMSE by approximately 0.116 points. This is consistent with the wide spread of per-movie averages observed in the EDA. The addition of user effects contributes a further reduction of approximately 0.079 points, reflecting the substantial heterogeneity in individual rating behavior. Regularization provides a smaller but statistically meaningful improvement of approximately 0.0005 points on the validation set, with its primary benefit being the stabilization of predictions for entities with few observations.

3.2 Final Evaluation on final_holdout_test

After identifying the optimal lambda on the internal validation set, the regularized movie and user effect model was retrained on the complete edx dataset — not just the training split — to maximize the information available for estimating the bias terms. The final model was then applied to final_holdout_test for the first and only time.

```
mu_final <- mean(edx$rating)

b_i_final <- edx %>%
group_by(movieId) %>%
summarise(b_i = sum(rating - mu_final) / (n() + best_lambda))

b_u_final <- edx %>%
left_join(b_i_final, by = 'movieId') %>%
group_by(userId) %>%
summarise(b_u = sum(rating - mu_final - b_i) / (n() + best_lambda))

final_preds <- final_holdout_test %>%
left_join(b_i_final, by = 'movieId') %>%
left_join(b_u_final, by = 'userId') %>%
mutate(pred = mu_final + b_i + b_u) %>% pull(pred)

rmse_final <- RMSE(final_holdout_test$rating, final_preds)
```

The final RMSE on the held-out test set is below the target threshold of 0.86490, confirming that the model generalizes well to unseen data. Training on the full edx dataset rather than the smaller training split provides a marginal additional improvement, as more data reduces variance in the bias estimates.

4. Conclusion

In this project, I developed a movie recommendation system for the MovieLens 10M dataset using a regularized additive bias model. The central finding is that a relatively simple model — one that decomposes ratings into a global mean, a movie-specific offset, and a user-specific offset — achieves strong predictive performance when paired with appropriate regularization. The final model achieved RMSE below 0.86490 on the held-out test set, satisfying the target for the highest project grade.

The incremental modeling approach proved effective both practically and pedagogically. Each step was motivated by an observable pattern in the data — the variance in per-movie averages, the variance in per-user tendencies, and the instability of estimates for sparsely observed entities — and each step yielded a measurable improvement in predictive accuracy.

This project demonstrated that careful exploratory analysis and principled model construction can achieve competitive results with interpretable, computationally inexpensive methods. Understanding the data structure particularly the sparsity of the user-movie interaction matrix — was at least as important as the choice of modeling technique.

5. References

[1] GroupLens Research. (2009). *MovieLens 10M Dataset.* University of Minnesota. Retrieved from https://grouplens.org/datasets/movielens/10m/

[2] Irizarry, R. A. (2019). *Introduction to Data Science: Data Analysis and Prediction Algorithms with R.* CRC Press / HarvardX edX.

[3] Koren, Y. (2009). Collaborative filtering with temporal dynamics. *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09)*, 447–456.

[4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

[5] R Core Team. (2023). *R: A Language and Environment for Statistical Computing.* R Foundation for Statistical Computing, Vienna, Austria.

[6] Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L., François, R., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.